Lab 8: Design and FPGA Implementation of Memory System with Address Decoding for a Single-Cycle RISC-V Processor

Objective

To design and implement the data memory and address decoding for a single-cycle RISC-V processor using Verilog HDL, verify functionality through simulation, synthesize the design on an FPGA, and demonstrate operation using switches and LEDs.

Proposed Memory Map

- Data Memory: 0–511
- LEDs: 512–767
- Switches: 768–1023

Task 1: Design of Data Memory and Address Decoder

Objective

To design a data memory module and an address decoder that routes CPU memory accesses to the correct memory or peripheral.

Address Decoding

Address decoding is the mechanism by which the processor determines which memory or peripheral device must respond to a given memory access. Since the CPU presents a single unified address bus, address decoding ensures that only one device is activated at a time based on the address value.

In this memory system, three address-mapped devices are present:

- Data Memory
- LED Output Interface
- Switch Input Interface

Although the processor generates a 32-bit (or 64-bit) memory address, only the 10 least significant bits of the address are used in this lab to form a local memory map. This allows addressing up to $2^{10} = 1024$ memory-mapped locations.

The address format is shown below:

```
address[9:0] = [Device Select (2 bits) | Local Address (8 bits)]
```

- `address[9:8]` selects which device is accessed.
- `address[7:0]` selects the specific register or memory location inside that device.

Device Selection

address[9:8]	Address Range	Device	Enable Signal
00	0–511	Data Memory	DataMemSelect
01	512–767	LEDs	LEDSelect
10	768–1023	Switches	SwitchSelect
11	–	Unused	None

Decoder Operation

The address decoder continuously monitors the CPU address and generates internal enable signals based on `address[9:8]`:

- When accessing Data Memory, the decoder enables memory read/write.
- When accessing LEDs, the decoder enables LED write only.
- When accessing Switches, the decoder enables switch read only.

Only one device is enabled at any time, preventing bus contention and ensuring correct routing of data.

Example

If the CPU issues the address:

$$\texttt{address = 0x210} = 528_{10}$$

$$\texttt{address[9:8] = 01}$$

The decoder enables the LED module, and the lower 8 bits (`address[7:0]`) determine which LED register is accessed.

Similarly, for:

$$\texttt{address = 0x350} = 848_{10}$$

$$\texttt{address[9:8] = 10}$$

The Switch module is selected.

This decoding scheme provides a simple, scalable, and hardware-efficient way to implement memory-mapped I/O in the single-cycle RISC-V processor.

Procedure

(a) Design Data Memory Module:

- Inputs: `clk`, `MemWrite`, `address`, `write_data`
- Output: `read_data`
- Implement as a 512×32 memory array.

(b) Use LED and Switch modules as provided in Lab 5.

(c) Design Address Decoder using `address[9:8]` to generate:

- DataMemWrite, DataMemRead
- LEDWrite
- SwitchReadEnable

(d) Draw a block diagram showing CPU, Address Decoder, Data Memory, LEDs, and Switches.

Deliverables

- Block diagram of memory system with address decoding.

Task 2: Verilog Implementation and Testbench Development

Objective

To implement the data memory and address decoder in Verilog and verify functionality through simulation.

Procedure

(a) Implement Data Memory and Address Decoder modules in Verilog. The top module, in direct communication with the processor, may appear as follows:

```
module addressDecoderTop(
    input clk, rst,
    input [31:0] address,
    input readEnable, writeEnable,
    input [31:0] writeData,
    input [15:0] switches,

    output [31:0] readData,
    output [15:0] leds
);
...
endmodule
```

(b) Integrate LED and Switch modules provided in Lab 5.

(c) Develop a testbench:

- Write to Data Memory and LEDs.
- Read from Data Memory and Switches.
- Verify that the address decoder enables only the correct module for each access.

(d) Run simulation, generate waveforms, and verify:

- Data Memory stores and outputs correct data
- LEDs reflect written values
- Switch values are read correctly

Deliverables

- Verilog source files: `DataMemory.v`, `AddressDecoder.v`
- Testbench file: `MemorySystem_tb.v`
- Simulation waveforms showing correct memory and peripheral access.

Task 3: FPGA Synthesis and RTL Analysis

Objective

To synthesize the memory system on the FPGA, analyze the RTL structure, and verify operation using provided switches and LEDs.

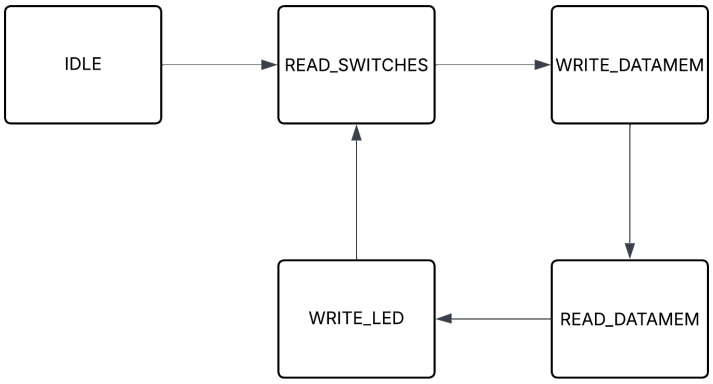
Procedure

(a) Integrate Data Memory, LEDs, Switches, and Address Decoder into a top-level design.

(b) Map memory and peripheral connections to FPGA pins.

(c) Test memory operations using switches or a simple FSM:

- Write to Data Memory and LEDs
- Read from Data Memory and Switches



(d) Observe LEDs to verify correct writes; confirm switch values are read properly.

(e) Synthesize the design and inspect the RTL schematic.

(f) Program the FPGA and verify memory-mapped operations interactively.

Deliverables

- RTL schematic screenshots
- Demonstration of FPGA setup showing correct memory and peripheral operation

Note: The switch interface, button debouncing module, and LED interface must be re used from Lab 5.

Assessment Rubric
Lab 8: Design and FPGA Implementation of Memory System with Address Decoding for a Single-Cycle RISC-V Processor

Name:	Student ID:	Section:

Points Distribution

	Task No.	LR 2 (Code)	LR 5 (Results)	LR 11 (Design)
In-Lab	Task 1	-	-	/20
	Task 2	/20	/15	-
	Task 3	/10	/15	-
Total Points		**/30**	**/30**	**/20**
CLO Mapped		CLO 1		

Affective Domain Rubric		Points	CLO Mapped
AR7	**Report Submission & Git Upload**	/10 & /10	CLO 1

CLO	Total Points	Points Obtained
1	100	
Total	**100**	

For description of different levels of the mapped rubrics, please refer to the Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics provided here.